

PRAKTIKUM SISTEM OPERASI

MODUL 6

Proses Sekuensial dan Konkuren



Oleh:

Reza Afiansyah Wibowo

2311104062

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. [10 Poin] Batasan maksimal proses pada Xinu ditentukan secara software melalui konstanta NPROC, yang dideklarasikan pada file konfigurasi sistem (umumnya pada system/conf.h atau include/kernel.h, tergantung struktur source tree Xinu yang dipakai). Konstanta ini menjadi ukuran array tabel proses “struct procent proctab[NPROC]”, sehingga jumlah maksimum proses yang bisa dibuat sama dengan nilai NPROC tersebut.

Pada konfigurasi Xinu standar yang umum dipakai di praktikum, nilai default NPROC adalah 100 proses. (Catatan: nilai ini perlu diverifikasi pada source tree masing-masing, karena dapat berbeda jika sudah dimodifikasi oleh asisten praktikum. Gunakan perintah berikut untuk mencarinya: `grep -rn “NPROC”` pada folder source Xinu.)

Untuk mengubah batas maksimal menjadi 150 proses, ubah baris deklarasi konstanta tersebut menjadi:

#define NPROC 150

kemudian lakukan kompilasi ulang dengan `make clean && make` agar ukuran tabel proses yang baru diterapkan.

2. [20 Poin] Pada `main_ex1`, `sndA()` dan `sndB()` dipanggil secara langsung sebagai pemanggilan fungsi biasa di dalam proses `main`, bukan dibuat sebagai proses Xinu baru melalui `create()`. Karena `sndA()` berisi perulangan `while(1)` yang tidak pernah berhenti atau `return`, eksekusi program akan terjebak selamanya di dalam `sndA()`, sehingga baris pemanggilan `sndB()` tidak akan pernah tercapai/dijalankan. Output yang dihasilkan hanya karakter A yang tercetak berulang setiap 1 detik, dan huruf B tidak akan pernah muncul:

A

A

A

... (berlanjut terus, karena `sndA` tidak pernah `return`)

Hal ini menunjukkan sifat eksekusi sekuensial yang blocking: proses/fungsi berikutnya hanya dapat berjalan setelah fungsi sebelumnya selesai (`return`), dan karena `sndA` tidak pernah selesai, `sndB` tidak pernah kebagian giliran.

3. [20 Poin] Pada `main_ex2`, `sndA` dan `sndB` dibuat sebagai dua proses Xinu yang berbeda melalui `create()`, kemudian diaktifkan dengan `resume()`, masing-masing dengan prioritas 20. Karena keduanya merupakan proses independen dan masing-masing memanggil `sleepms(1000)` (yang membuat proses memberi giliran/memicu penjadwalan ulang), scheduler Xinu menjalankan kedua proses tersebut secara konkuren dan bergantian.

Output yang dihasilkan menampilkan huruf A dan B yang muncul berselang-seling, kurang lebih setiap 1 detik:

A

B

A

B

...

Urutan persis antara A dan B pada tiap siklus dapat sedikit berbeda tergantung penjadwalan, namun kedua huruf tetap muncul secara bergantian karena dijalankan sebagai dua proses terpisah yang berjalan konkuren.

4. [20 Poin] Pada `main_ex3`, satu fungsi yang sama, `sndChar(char c)`, digunakan untuk membuat dua proses berbeda melalui `create()`, masing-masing diberi argumen karakter yang berbeda (A dan B). Meskipun kode fungsinya identik, setiap proses memiliki stack dan konteks eksekusi (PCB) sendiri, sehingga nilai parameter `c` pada masing-masing proses berbeda dan tidak saling memengaruhi satu sama lain. Kedua proses berjalan konkuren (sama seperti `main_ex2`), sehingga output yang dihasilkan juga menampilkan karakter A dan B yang muncul bergantian setiap kurang lebih 1 detik:

A

B

A

B

...

Hasil ini membuktikan bahwa proses-proses yang dibuat dari fungsi yang sama tetap berjalan independen dan konkuren, karena masing-masing memiliki context (stack, register, program counter) tersendiri yang dikelola lewat PCB masing-masing proses, terlepas dari kode programnya identik.

5. [30 Poin] Jika produser dan konsumen dijalankan sebagai dua proses konkuren tanpa mekanisme sinkronisasi apa pun, hasil keluarannya tidak akan berupa urutan rapi 1, 2, 3, ..., 1000 seperti intuisi awal. Hasilnya sangat bergantung pada bagaimana scheduler Xinu menjalankan kedua proses tersebut secara bergantian. Beberapa kemungkinan hasil yang dapat muncul:
- Konsumer mencetak nilai 0 sebanyak 1000 kali, jika proses konsumer mendapat giliran berjalan hingga selesai sebelum produser sempat menambah nilai `n` sama sekali.
 - Konsumer mencetak nilai 1000 sebanyak 1000 kali, jika sebaliknya produser sudah selesai menambah `n` hingga 1000 sebelum konsumer mulai mencetak.
 - Konsumer mencetak kombinasi nilai yang melompat-lompat atau berulang (misalnya beberapa baris bernilai sama secara berturut-turut, lalu melompat ke nilai

yang lebih besar), jika scheduler berpindah-pindah di antara kedua proses pada titik yang tidak terduga.

Penyebab utama dari hasil yang tidak sesuai intuisi ini:

1. Variabel n bersifat global dan diakses bersama (shared) oleh dua proses tanpa adanya mekanisme sinkronisasi (seperti semaphore atau mutex) yang mengatur urutan akses terhadap variabel tersebut, sehingga terjadi race condition.
2. Xinu menjadwalkan proses berdasarkan prioritas dan status ready queue, bukan berdasarkan urutan logis program yang ditulis programmer. Selama proses tidak melakukan operasi yang memicu penjadwalan ulang (seperti sleep atau menunggu semaphore), proses tersebut dapat berjalan hingga selesai (run-to-completion) tanpa diselingi proses lain.
3. Karena tidak ada titik sinkronisasi antara produser dan konsumen, urutan eksekusi keduanya menjadi non-deterministik, sehingga hasil program dapat berbeda-beda setiap kali dijalankan meskipun kodenya sama.

Hasil ini menunjukkan pentingnya sinkronisasi antar-proses (misalnya menggunakan semaphore) ketika dua atau lebih proses mengakses dan memodifikasi data bersama secara konkuren, agar hasil program dapat diprediksi dan tidak bergantung pada timing eksekusi yang acak.